

AEGISPAY

Agentic Commerce Architecture V4

The protocol-aware Trust & Growth Layer for Agentic Commerce. AI proposes. AegisPay deterministically validates, authorizes and controls. Razorpay executes only approved actions.

Different protocols can enter AegisPay, but none can bypass the AegisPay Control Plane.

1. What is AegisPay?

AegisPay sits between AI agents and payment providers. This V4 adds the modern agentic-commerce protocol ecosystem as a single, unified entry point: every protocol enters through one gateway and collapses into one normalized intent before the control plane.

Pillar	What it means
GROW	AI helps a merchant increase revenue (upsell, cross-sell, bundles, campaigns, experiments).
SELL	AI buyers understand intent, discover, recommend, build carts, get authorization and pay.
PROTECT	AI can reason and recommend, but never directly controls money.

2. GROW / SELL / PROTECT



Figure 1 - Full system: external protocols -> Protocol Gateway -> Control Plane -> providers

Protocols arrive on the left as adapters; they become one normalized intent; the control plane gates it; only then does a provider move money. There is exactly one money path.

3. The Protocol Gateway



Figure 2 - Every protocol enters through the gateway and collapses into one normalized intent

A single **Protocol Gateway** normalizes all external protocols into the **canonical AegisPay intent**. The core never imports a protocol SDK. A new protocol is a new adapter - it changes the transport, never the money path.



Figure 3 - Each adapter authenticates, validates, normalizes and scope-checks before the control plane

4. Protocol ecosystem & maturity

Protocol	Role in AegisPay	Maturity	Honest note
MCP	controlled agent tools / context	Core	implemented adapter; safe tools only
A2A	agent-to-agent communication	Adapter-ready	agent endpoint over the canonical model
UCP	commerce interoperability	Adapter-ready	map to canonical commerce actions
ACP	agentic checkout / commerce	Adapter-ready	message & card mapping
AP2	payment mandates & verifiable authorization	Experimental	map where semantics align; no claim
x402	machine / pay-per-use payments	Experimental	route through a normal provider-funded action
A2UI	agent-driven UI	Experimental	watch-list; no claim
NPCI UAP / UPI	India agentic-payment ecosystem	Future	watch; no compliance claim

Maturity labels are honest: **Core** is implemented, **Adapter-ready** has a mapping, **Experimental** maps only where semantics align, and **Future** is a watch-list with no compliance claim. We never state a protocol is supported unless the adapter actually normalizes it.

5. Component architecture

Component	What it does	Fails safely by
Protocol Gateway	authenticate + validate + normalize external protocols	rejects anything it can't normalize
Commerce Orchestrator	runs the purchase journey	only allowed state transitions
Policy Engine	fixed limits (amount, category, hours)	denies if unsure
Risk Engine	explainable score + why	escalates on doubt
Authorization Engine	binds consent to one transaction	expires, single-use
Payment Engine	provider-abstract lifecycle (Razorpay / UPI / x402)	returns UNKNOWN, never guesses
Human Approval	person decides high value	scoped, expiring, non-reusable
Webhook Gateway	verifies + dedupes provider events	drops bad/duplicate events
Reconciliation	resolves UNKNOWN from provider truth	never retries blindly
Audit + Passport	records every decision	tamper-evident chain

6. AI / Control-plane trust boundary



Figure 4 - The AI thinks; the control plane decides and acts

The AI runtime has **no payment keys, no database credentials, no unrestricted money tools and no policy mutation capability**. A manipulated agent can only produce an intent the control plane gates, regardless of which protocol it arrived through.

7. SELL flow

Intent -> catalog -> recommendation -> cart -> authorization -> payment -> verified webhook -> reconciliation -> Passport. Detail in the SELL document.

8. GROW flow

Data -> opportunity -> proposal -> rules -> estimate -> approval -> budget -> execution -> A/B -> incremental measurement -> learn. Detail in the GROW document.

9. Protocol-specific security

Concern	How it is handled
Authentication	OAuth / client-credential per protocol, mapped to a canonical identity
Agent identity	protocol subject -> canonical agent_id; no identity confusion across protocols
Schema validation	strict typed schema on the normalized intent; free text never trusted
Replay protection	single-use nonce, expiry, binding to a transaction digest
Scoped authorization	one action, one amount, one policy version; expiring
Idempotency	key per command; designed to prevent duplicate financial effects
Tool allowlisting	only safe tools exposed; no money tool to any protocol
Tenant isolation	tenant resolved from auth; RLS at the database

10. Policy / Risk / Authorization

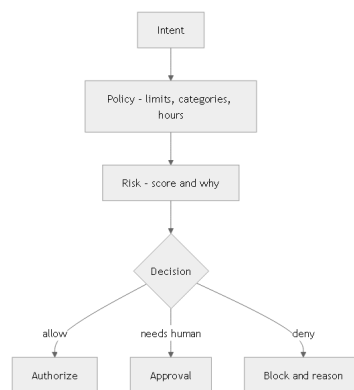


Figure 5 - Every money action passes a deterministic gate

- **Policy** is versioned and merchant-owned; the AI cannot change it.
- **Risk** is explainable: a score plus the reasons.
- **Authorization** is scoped, transaction-bound, expiring and single-use.

11. Human approval

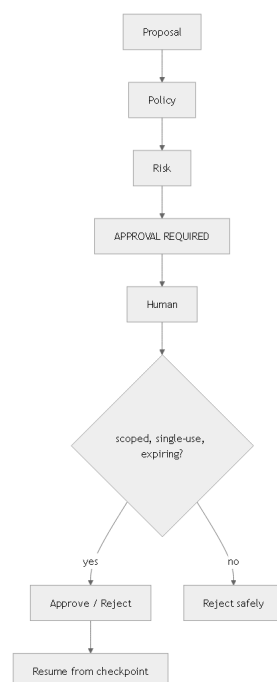


Figure 6 - A proposal escalates to a person only when required, then resumes

An approval is scoped to this exact action, single-use, transaction-bound, expiring and immutable after issuance - it cannot be replayed or stretched into a bigger purchase.

12. Payment state machine

The full lifecycle with failure/recovery states. **UNKNOWN** is a **first-class state** and exits only on provider truth. We never blindly retry an UNKNOWN payment.

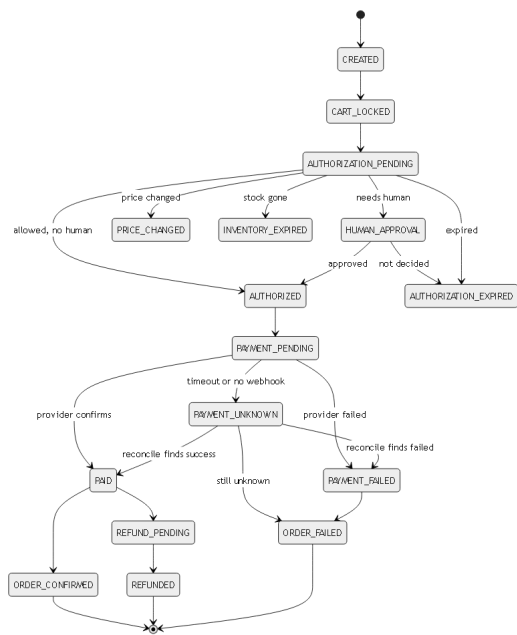


Figure 7 - Complete payment state machine, including failure and recovery states

13. Webhook + idempotency

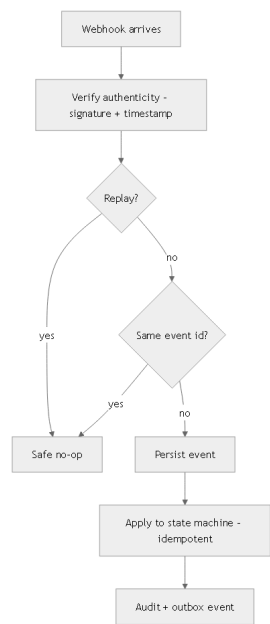


Figure 8 - Webhooks are untrusted until verified and deduped; duplicates are safe no-ops

Command	Idempotency scope	Effect
create order	order key + request hash	same request returns the same order
payment operation	payment key	retry returns the prior provider result
refund	(payment, refund key)	one effective refund per key
webhook processing	(provider, event id)	duplicates are safe no-ops
campaign budget	(campaign, spend request)	atomic reservation; no double-spend

Every financial command carries an idempotency key. This is **designed to prevent duplicate financial effects** - we do not claim duplicate payments are impossible.

14. Transactional Outbox

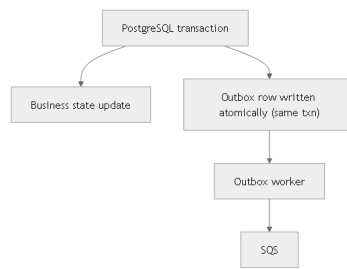


Figure 9 - A state change and its event are written in one transaction

Because the state update and the outbox event commit together, you cannot have a state change with no event, or an event with no state change. A worker publishes to SQS at-least-once, and consumers are idempotent.

15. Reconciliation

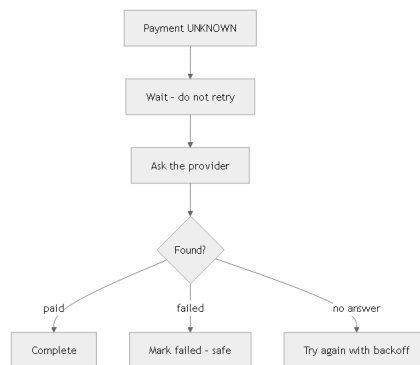


Figure 10 - UNKNOWN payments are checked, never blindly retried

A timed-out or unknown payment is never re-created. We wait, ask the provider, then complete or fail safely, with backoff and escalation.

16. Cart / Price / Inventory protection

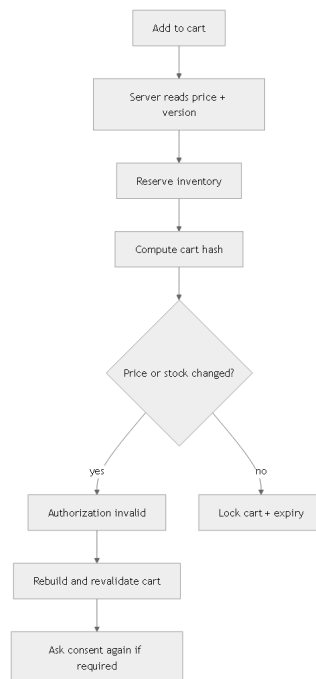


Figure 11 - If price, version or stock changes, authorization becomes invalid

Guard	What it does
Server-owned prices	price from the DB, never the AI or client
Cart hash	snapshot; a change invalidates authorization
Price version	a price change is detected and checkout re-validated
Inventory reservation	stock is held so it can't oversell
Cart expiry	a stale cart cannot be paid
Product ownership lock	a product belongs to one merchant; cross-merchant reference rejected

17. Refund architecture

Refunds are controlled. The AI has **no** unrestricted refund tool - a refund needs policy, authorization and provider verification.

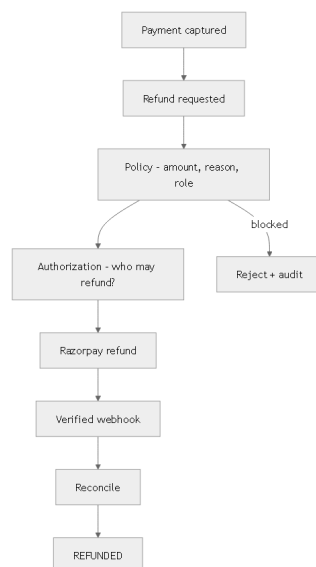


Figure 12 - Refund flow gated by policy and authorization

18. Agent security

Threat	Prevention
Prompt injection	catalog is data, not instructions; intent schema-validated
Tool abuse	only safe tools exposed; no money tool
Agent impersonation	signed credentials, session binding, revocation
Credential theft	hashed, rotatable, scoped keys
Authorization replay	transaction-bound, single-use, expiring authz
Self-escalation	no tool to change own limits/policy
Malicious catalog / input	typed, validated, injection classifier

19. Multi-tenancy + RLS

Shared PostgreSQL, shared schema, Row-Level Security. Two DB roles: an application role that cannot bypass RLS, and a migration/owner role for schema only.

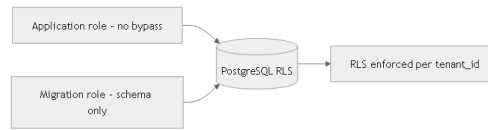


Figure 13 - Application role respects RLS; migration role only changes schema

20. Audit + Transaction Passport



Figure 14 - Events are linked by hash into a tamper-evident, anchored chain

Append-only, RLS read-only, hash-chained and anchored to an immutable root. The Transaction Passport bundles the who/what/why/approval/provider. It is **tamper-evident**, not tamper-proof.

21. AWS deployment

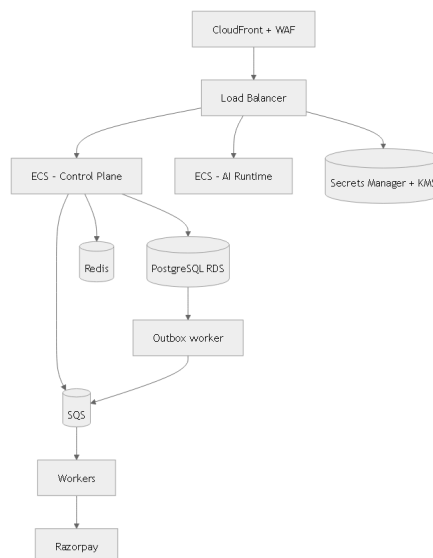


Figure 15 - Two ECS services, managed data, one queue

CloudFront + WAF -> ALB -> ECS (Control Plane + AI Runtime). PostgreSQL RDS, Redis, SQS, Secrets Manager + KMS. No Kubernetes - not needed at this scale.

22. Observability + SLOs

Dashboards cover the money path, the decision path and AI health. Logs carry correlation, tenant, transaction and agent ids; secrets and PII are never logged.

Signal	What it shows
Payment success / UNKNOWN rate	provider finality and ambiguity
Webhook latency / duplicate rate	ingest health
Policy denial / risk escalation	decision guardrails
Human approval latency	time to a decision
Reconciliation backlog	stuck UNKNOWN
AI tool failure / agent completion	agent reliability
Campaign ROI	growth effectiveness

Realistic SLOs: API availability ~99.9% (honest, single-region multi-AZ); RPO <= 15 min; RTO <= 1 hour; payment state transition typically < 2 s; webhook processing within seconds; reconciliation auto-resolves >= 95% of UNKNOWN.

23. Disaster recovery

Multi-AZ, PITR backups (RPO <= 15 min), quarterly restore drills, documented RTO <= 1 hour. Audit integrity stays independent of DB restore via the hash chain + S3 anchor.

24. Threat model

Threat	Attack surface	Impact	Prevent	Detect	Recover
Prompt injection	catalog, prompts	unauthorized spend	DATA-only, schema, allowlist	injection + denial alerts	block, revoke, audit
Tool abuse	agent tool calls	misuse	allowlist, rate limit, budget	anomaly alerts	throttle, suspend
Agent impersonation	credentials	spend as another agent	signed creds, mTLS, session binding	auth anomalies	revoke, re-issue
Protocol spoofing	gateway adapters	false intent	authenticate + schema validate	adapter anomalies	reject, reconcile
Webhook spoofing	webhook endpoint	false state	signature + timestamp verify	signature failure alerts	ignore, reconcile
Replay attack	authz / webhook / approval	repeat an action	single-use, dedupe, nonce	replay detectors	reject
Duplicate payment	retries	double effect	idempotency keys	duplicate detectors	return prior result
Price manipulation	cart / intent	wrong amount	server-owned prices + hash	price mismatch	reject
Tenant escape	queries	cross-tenant leak	RLS + tenant context + app role	isolation tests	block + audit
Compromised AI runtime	LLM service	attempt money path	no secrets, no money tools, allowlist	red-team + anomalies	quarantine

25. Failure testing

Fault	Detection	Safe state	Recovery	Audit
Razorpay timeout	long call / missing webhook	PAYMENT_UNKNOWN	reconcile -> complete/fail	payment.unknown / reconciled
Duplicate webhook	same event id	no-op	acknowledge only	webhook.deduped
Invalid webhook	signature check	reject, no state	alert + ignore	webhook.failed
Price changed in checkout	price version mismatch	authorization invalid	revalidate, ask consent	cart.rejected
Inventory unavailable	reservation fails	cart not locked	re-select / notify	inventory.reserve_failed
Protocol adapter rejects	schema/auth failure	no intent produced	return typed error	gateway.rejected
AI unavailable	timeout	agent path degrades	control plane still reconciles	agent.unavailable
Policy violation	policy decision	DENY, no payment	record reason	policy.denied
Expired approval	approval timer	expire, no action	re-request approval	approval.expired
DB / outbox failure	publish error	transaction rolls back	retry from outbox	outbox.retry
Campaign budget exhaustion	atomic check	deny / pause	no spend	campaign.budget_exceeded

26. Production readiness checklist

Area	Present
AI	No money path, tool allowlist, structured output, policy + risk, human approval, red-team
Protocols	Single gateway, adapters normalize to one intent, protocol security, honest maturity
Payments	Idempotency, full state machine, webhook verify + dedupe, reconciliation, refund caps
Trust	Versioned policy, explainable risk, scoped/expiring authz, Transaction Passport, outbox
Growth	Atomic budget ledger, kill switch, A/B + incremental measurement
Security	Auth (user, agent, service), secrets isolation, RLS app role, encryption, rate limits
Reliability	Timeouts, retries, circuit breakers, outbox + DLQ, backups + DR, SLOs
Observability	Metrics, logs with IDs, tracing, alerts, dashboards

Why AegisPay is different

Different protocols can enter AegisPay, but none can bypass the AegisPay Control Plane.

- **One control plane.** Every protocol resolves to a normalized intent that the control plane gates deterministically.
- **No direct money path.** The AI runtime has no payment keys, no DB credentials, no unrestricted money tools.
- **Deterministic authorization.** Policy, risk and a scoped, expiring authorization gate every action.
- **Provider truth.** Payment finality comes from verified webhooks or reconciliation, never a guess.
- **Escape hatch free.** A new protocol is a new adapter over the same canonical model — it cannot introduce a new money path.
- **Tamper-evident.** Every decision is recorded in a hash-chained audit ledger and Transaction Passport.